

PA1 – Joseph Cassarino – CMSC 471 SEC 02

Q1 – Heuristics:

Steps:

1. c = number of slots (letters) that are mismatched between state and goal.
2. A legal action alters at most one character within a word at a time; if a state differs from the goal in n positions, any valid path from state to goal must perform at least n substitutions to correct each mismatched character.
3. Every action costs at least 1. Every action can only solve 1 mismatched character. There is the very high likelihood that the optimal path will include detours or intermediate steps so the true optimal remaining steps is $\geq n$.

Scrabble:

1. c is the sum of the scrabble values of the goal letters for all mismatched slots between state and goal.
2. A legal action only changes one character at a time, and its cost is based entirely on the newly inserted letter. For each mismatched position, the target letter from the goal word has to be placed into that slot at least once before reaching the goal, so you are forced to pay at least that letter's Scrabble value.
3. Every inserted letter costs at least 1, and each action can only place one goal letter. Any intermediate words will add costs on top of the final swap, so the true optimal remaining cost is $\geq c$.

Frequency:

1. c = number of slots (letters) that are mismatched between state and goal.
2. Because each move costs $1 + \text{rarity}(w)$ and all rarity values in the dictionary are ≥ 0 the absolute cheapest possible move still costs at least 1. If a state differs from the goal in n positions, you need at least n substitutions to fix them, each costing at minimum 1.
3. Every action costs at least 1. Every action can fix at most 1 mismatched character. Any intermediate words will add costs above 1 per step, meaning the true optimal remaining cost is $\geq n$.

Q2 – A* vs. UCS:

Word Pair	Cost Measure	UCS Cost	UCS Nodes	A* Cost	A* Nodes	Reduction	Heuristic Cost
Why->Ask	Steps	5	773	5	16	97.93%	3
Why->Ask	Scrabble	9	765	9	13	98.3%	7
Why->Ask	Frequency	38.86	808	38.86	755	6.56%	3
Bear->Duck	Steps	4	691	4	6	99.13%	3
Bear->Duck	Scrabble	11	2981	11	6	99.8%	11
Bear->Duck	Frequency	38.24	1084	38.24	720	33.58%	3

1. Yes, despite lots of variance in their nodes expanded, they both always arrived at the same optimal cost.
2. Both word pairs experienced the highest reduction when using scrabble.
3. The frequency heuristic was by far the least informative in both word pairs.
4.
 - a. Scrabble is the most informative because it is able to utilize the most information out of the heuristics. It can capture the actual costs of the letters needed to reach the goal, so its estimate is much tighter to the true remaining cost.
 - b. Steps is the 2nd most informative; each action costs strictly 1, so estimating 1 per mismatch is the closest we can get to the true step distance. There are no letter or word-specific weights tipping the scale on how much an action costs, there is only so much it can vary based on detours or intermediate words.
 - c. The actual cost to transition words under the frequency cost model is completely thrown around by word rarity values, which cannot be accurately accounted for in our heuristic; they can be as low as 0.01 or as high as 16. Given the fact that we *could* get a word pair with a lucky path of all near-0 rarities we can't let our heuristic go any higher than just "# of steps" while the actual cost vastly increases because it's usually not that.

Q3 - Reversibility:

Steps:

1. Yes
2. The number of steps in a single action will always be one, and we are solely counting steps as the cost indicator.

Scrabble:

1. No
2. Reversing the process means the character you're swapping into your state word is different.
3. If $w_1 = \text{cat}$ and $w_2 = \text{cap}$: Going from $w_1 \rightarrow w_2$ costs 3 because a 'p' is being swapped in. Going from $w_2 \rightarrow w_1$ costs 1 because a 't' is being swapped in.

Frequency:

1. No
2. Frequency based cost depends on the destination word. If the order is reversed then your destination word is different.
3. If $w_1 = \text{cat}$ and $w_2 = \text{cap}$: $w_1 \rightarrow w_2$ costs $1 + 6.052741$. $w_2 \rightarrow w_1$ costs $1 + 6.886906$

Q4 – LLM Audit:

1 & 2:

Claude Sonnet 4.6 (low) was fed the prompt:

- “In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost”

And proposed this heuristic:

Rarity-Aware Admissible Heuristic for DOGCAT

Setup and Notation

Let:

- n = current state (a word)
- g = goal state
- $d(n, g)$ = number of positions where n and g differ (this is $h_0(n)$)
- $\text{rarity}(w)$ = the nonnegative rarity value of word w stored in the dictionary
- $\text{cost}(w) = 1 + \text{rarity}(w)$ = the cost paid upon *entering* word w

Note that reaching the goal requires entering it, so the goal's own rarity is always paid.

Proposed Heuristic

$$h_1(n) = d(n, g) + \text{rarity}(g)$$

In words: the number of differing positions plus the rarity of the goal state itself.

Precise Definition

For every state n :

$$h_1(n) = |\{i : n[i] \neq g[i]\}| + \text{rarity}(g)$$

where the first term counts mismatched positions and $\text{rarity}(g) \geq 0$ is a fixed constant throughout the search.

3:

I believe this heuristic to be admissible: Any path from n to g must make at least $d(n, g)$ swap actions, and each action can correct at most one position and the positions are independent. Each step takes us to some intermediate word, each costing at least 1. This gives a contribution of at least $d(n, g)$ to the remaining cost. Also, any path from n to g must also enter the goal word (g) exactly once, guaranteeing that it will incur the cost of $1 + \text{rarity}(g)$.

We do not need to evaluate the case in which $n == g$, because in that state a heuristic is not even needed. Any solution path from n visits at least 'k' words after 'n', the last of those visited being 'g'. The total remaining cost is:

$$k * 1 + \text{rarity}(g)$$

The first term comes from the fact that at least k words are entered, each costing at least 1. The second term comes from the fact that g is the last of those k words, and its entry cost is $1 + \text{rarity}(g)$, meaning it contributes an additional $\text{rarity}(g)$ on top of the 1 already counted in the $k * 1$ term.

Therefore $C^*(n) \geq d(n, g) + \text{rarity}(g) = h_1(n)$, so h_1 never overestimates the true optimal remaining cost. h_1 is admissible.

5: Using this heuristic, why->ask gets an estimated cost of $3 + 4.2326436 = 7.2326436$. Bear->duck gets an estimated cost of $4 + 7.2297187 = 11.2297187$. These are marginally more informative. When running the tests again with this frequency heuristic, the cost doesn't change at all, and neither does the amount of nodes expanded by UCS. However A^* expanded 121 less nodes for why->ask, and 418 less for bear->duck. This shows there would be some benefit in search efficiency when adopting this heuristic.